

## ADS EXP8: SMOTE (Synthetic Minority Oversampling Technique) for Imbalanced Classification

---

- Use SMOTE technique to generate synthetic data.(to solve the problem of class imbalance)

```
In [10]: from imblearn.over_sampling import SMOTE
          from matplotlib import pyplot
          from numpy import where
          import pandas as pd

          df = pd.read_csv('Churn_Modelling.csv')
          df.head()
```

Out[10]:

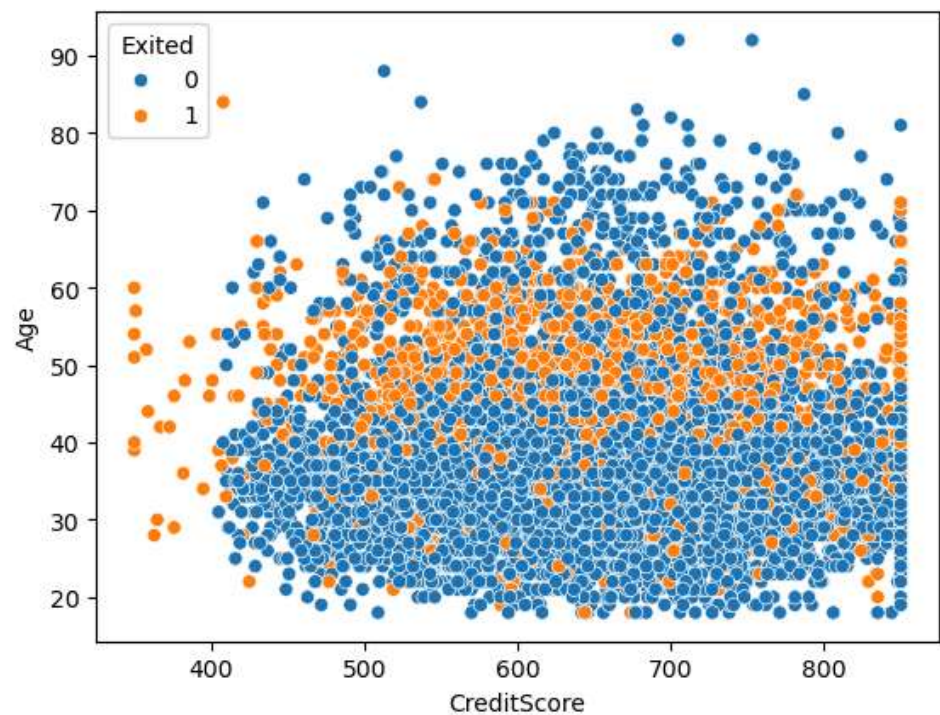
|   | RowNumber | CustomerId | Surname  | CreditScore | Geography | C |
|---|-----------|------------|----------|-------------|-----------|---|
| 0 | 1         | 15634602   | Hargrave | 619         | France    | F |
| 1 | 2         | 15647311   | Hill     | 608         | Spain     | F |
| 2 | 3         | 15619304   | Onio     | 502         | France    | F |
| 3 | 4         | 15701354   | Boni     | 699         | France    | F |
| 4 | 5         | 15737888   | Mitchell | 850         | Spain     | F |

```
In [11]: import seaborn as sns

data = df[['CreditScore', 'Age', 'Exited',]]
print(data.head(10))
sns.scatterplot(data = data, x = 'CreditScore', y = 'Age', hue =
'Exited')
```

|   | CreditScore | Age | Exited |
|---|-------------|-----|--------|
| 0 | 619         | 42  | 1      |
| 1 | 608         | 41  | 0      |
| 2 | 502         | 42  | 1      |
| 3 | 699         | 39  | 0      |
| 4 | 850         | 43  | 0      |
| 5 | 645         | 44  | 1      |
| 6 | 822         | 50  | 0      |
| 7 | 376         | 29  | 1      |
| 8 | 501         | 44  | 0      |
| 9 | 684         | 27  | 0      |

Out[11]: <Axes: xlabel='CreditScore', ylabel='Age'>



```
In [12]: # This code converts every text/categorical column in df into numbers.
from sklearn.preprocessing import LabelEncoder
for col in df.columns:
    if df[col].dtype == 'O': # Check if the column is of type object
        (text)
        label_encode = LabelEncoder()
        df[col] = label_encode.fit_transform(df[col])
df
```

Out[12]:

|      | RowNumber | CustomerId | Surname | CreditScore | Geography |
|------|-----------|------------|---------|-------------|-----------|
| 0    | 1         | 15634602   | 1115    | 619         | 0         |
| 1    | 2         | 15647311   | 1177    | 608         | 2         |
| 2    | 3         | 15619304   | 2040    | 502         | 0         |
| 3    | 4         | 15701354   | 289     | 699         | 0         |
| 4    | 5         | 15737888   | 1822    | 850         | 2         |
| ...  | ...       | ...        | ...     | ...         | ...       |
| 9995 | 9996      | 15606229   | 1999    | 771         | 0         |
| 9996 | 9997      | 15569892   | 1336    | 516         | 0         |
| 9997 | 9998      | 15584532   | 1570    | 709         | 0         |
| 9998 | 9999      | 15682355   | 2345    | 772         | 1         |
| 9999 | 10000     | 15628319   | 2751    | 792         | 0         |

10000 rows × 6 columns



```
In [13]: from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
#Splitting the data
from sklearn.tree import DecisionTreeClassifier
X_train, X_test, y_train, y_test =
train_test_split(df.drop('Exited',axis=1), df['Exited'], test_size =
0.2, random_state = 101)
```

```
In [14]: clf = DecisionTreeClassifier()
clf.fit(X_train,y_train)
y_pred=clf.predict(X_test)
print(classification_report(y_test, y_pred))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.87      | 0.84   | 0.85     | 1577    |
| 1            | 0.47      | 0.53   | 0.50     | 423     |
| accuracy     |           |        | 0.77     | 2000    |
| macro avg    | 0.67      | 0.68   | 0.68     | 2000    |
| weighted avg | 0.78      | 0.77   | 0.78     | 2000    |

```
In [15]: smote = SMOTE(sampling_strategy='auto',k_neighbors=5,random_state = 101)
X_oversample, y_oversample = smote.fit_resample(X_train, y_train)
```

```
In [16]: clf.fit(X_oversample,y_oversample)
y_predo=clf.predict(X_test)
print(classification_report(y_test, y_predo))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.89      | 0.77   | 0.82     | 1577    |
| 1            | 0.42      | 0.63   | 0.51     | 423     |
| accuracy     |           |        | 0.74     | 2000    |
| macro avg    | 0.65      | 0.70   | 0.67     | 2000    |
| weighted avg | 0.79      | 0.74   | 0.76     | 2000    |

```
In [17]: classifier = LogisticRegression()
classifier.fit(X_train, y_train)

print(classification_report(y_test, classifier.predict(X_test)))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.79      | 1.00   | 0.88     | 1577    |
| 1            | 0.00      | 0.00   | 0.00     | 423     |
| accuracy     |           |        | 0.79     | 2000    |
| macro avg    | 0.39      | 0.50   | 0.44     | 2000    |
| weighted avg | 0.62      | 0.79   | 0.70     | 2000    |

```
c:\ProgramData\anaconda3\Lib\site-
packages\sklearn\linear_model\_logistic.py:465:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max\_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>  
Please also refer to the documentation for alternative solver options:

```
https://scikit-
learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(
c:\ProgramData\anaconda3\Lib\site-
packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set
to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
c:\ProgramData\anaconda3\Lib\site-
packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set
to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
c:\ProgramData\anaconda3\Lib\site-
packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set
to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
```

```
In [18]: classifier.fit(X_oversample, y_oversample)
print(classification_report(y_test, classifier.predict(X_test)))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.82      | 0.53   | 0.64     | 1577    |
| 1            | 0.25      | 0.58   | 0.35     | 423     |
| accuracy     |           |        | 0.54     | 2000    |
| macro avg    | 0.53      | 0.55   | 0.49     | 2000    |
| weighted avg | 0.70      | 0.54   | 0.58     | 2000    |

```
c:\ProgramData\anaconda3\Lib\site-
packages\sklearn\linear_model\_logistic.py:465:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max\_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

[https://scikit-](https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression)

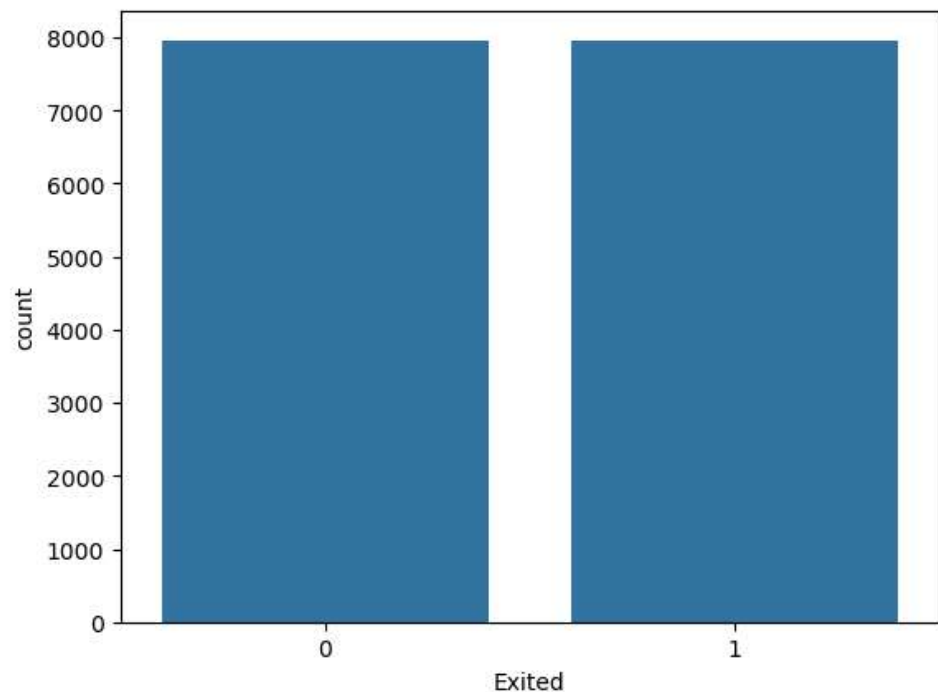
[learn.org/stable/modules/linear\\_model.html#logistic-regression](https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression)  
n\_iter\_i = \_check\_optimize\_result(

```
In [19]: smote = SMOTE(random_state = 101)
X, y = smote.fit_resample(df[['CreditScore', 'Age']], df['Exited'])
#Creating a new Oversampling Data Frame
df_oversampler = pd.DataFrame(X, columns = ['CreditScore', 'Age'])
df_oversampler['Exited']=y
print(df_oversampler.head())

sns.countplot(data=df_oversampler,x='Exited')
```

|   | CreditScore | Age | Exited |
|---|-------------|-----|--------|
| 0 | 619         | 42  | 1      |
| 1 | 608         | 41  | 0      |
| 2 | 502         | 42  | 1      |
| 3 | 699         | 39  | 0      |
| 4 | 850         | 43  | 0      |

Out[19]: <Axes: xlabel='Exited', ylabel='count'>



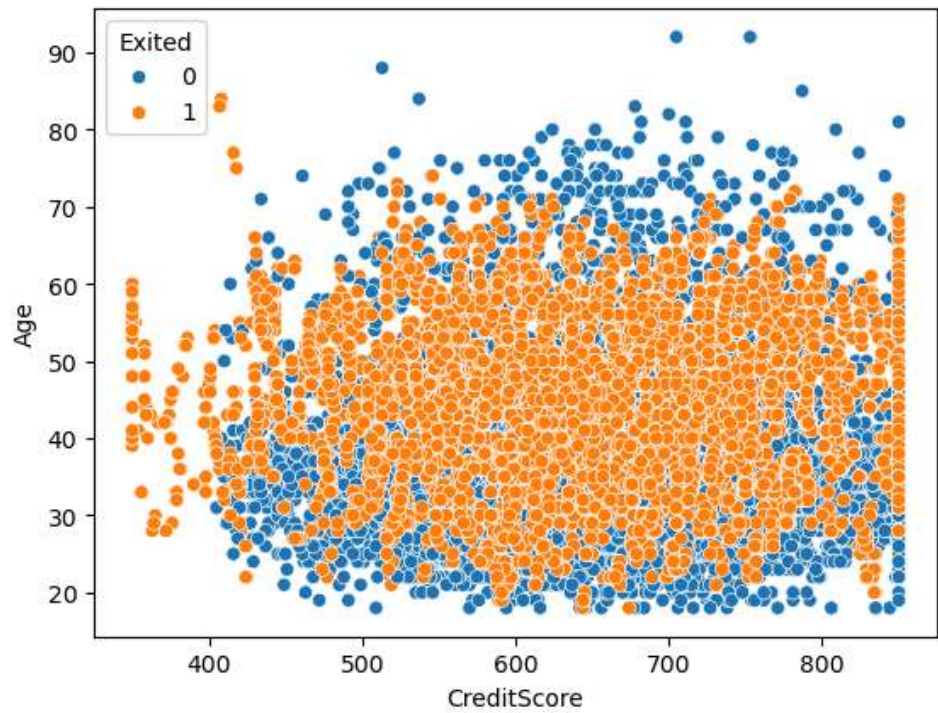
```
In [20]: from collections import Counter
X=df[['CreditScore', 'Age']]
y=df['Exited']

oversample = SMOTE()
X, y = oversample.fit_resample(X, y)
# summarize the new class distribution
counter = Counter(y)
print(counter)
```

Counter({1: 7963, 0: 7963})

```
In [21]: sns.scatterplot(data = df_oversampler, x = 'CreditScore', y = 'Age', hue  
= 'Exited')
```

```
Out[21]: <Axes: xlabel='CreditScore', ylabel='Age'>
```



---

Exported with [runcell](https://runcell.dev) — convert notebooks to HTML or PDF anytime at [runcell.dev](https://runcell.dev).